

ANEXO

Capturas de los Query's ejecutados

Ejercicio 1

PostgreSQL

The screenshot shows a PostgreSQL IDE with three tabs: "<postgres> Script-34", "<postgres> Week-4-Lab.sql", and "*<postgres> Script-35". The active tab is "Script-35", which contains the following SQL script:

```
-- =====  
-- EXPERIMENTO 1 - POSTGRESOL  
-- =====  
  
DROP TABLE IF EXISTS practica_oltp;  
  
CREATE TABLE practica_oltp (  
    id BIGSERIAL PRIMARY KEY,  
    nombre TEXT,  
    saldo NUMERIC(10,2),  
    created_at TIMESTAMP DEFAULT now()  
);  
  
-- Insertar 10 millones de registros  
INSERT INTO practica_oltp (nombre, saldo)  
SELECT  
    'Cliente ' || g,  
    random()*10000  
FROM generate_series(1,10000000) g;  
  
ANALYZE practica_oltp;  
  
-- Consulta puntual por PK  
EXPLAIN (ANALYZE, BUFFERS)  
SELECT * FROM practica_oltp WHERE id = 5000000;  
  
-- Nota esperada:  
-- Index Scan using oltp_test_pkey
```

Below the script, the "Estadísticas 1" (Statistics 1) window is open, showing the execution details of the highlighted INSERT query:

Name	Value
Updated Rows	10000000
Execute time	2m 14s
Start time	Mon Mar 02 23:40:21 CST 2026
Finish time	Mon Mar 02 23:42:36 CST 2026
Query	INSERT INTO practica_oltp (nombre, saldo) SELECT 'Cliente ' g, random()*10000 FROM generate_series(1,10000000) g

<postgres> Script-34

<postgres> Week-4-Lab.sql

*<postgres> Script-35 X

```

-- EXPERIMENTO 1 - POSTGRESOL
=====

DROP TABLE IF EXISTS practica_oltp;

-- CREATE TABLE practica_oltp (
--   id BIGSERIAL PRIMARY KEY,
--   nombre TEXT,
--   saldo NUMERIC(10,2),
--   created_at TIMESTAMP DEFAULT now()
-- );

-- Insertar 10 millones de registros
INSERT INTO practica_oltp (nombre, saldo)
SELECT
  'Cliente ' || g,
  random()*10000
FROM generate_series(1,10000000) g;

ANALYZE practica_oltp;

-- Consulta puntual por PK
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM practica_oltp WHERE id = 5000000;

-- Nota esperada:
-- Index Scan using oltp_test_pkey
-- Heap Fetch requerida (doble acceso)

```

Resultados 1 X

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM practica_oltp WHERE

Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

A-Z QUERY PLAN

1	Index Scan using practica_oltp_pkey on practica_oltp (cost=0.43..8.45 rows=1 width=37) (actual time= 1.417..1.419 rows=1 loops=1)
2	Index Cond: (id = 5000000)
3	Buffers: shared hit=4 read=3
4	Planning:
5	Buffers: shared hit=28
6	Planning Time: 11.436 ms
7	Execution Time: 5.838 ms

Mysql

The screenshot displays a MySQL IDE interface with three tabs at the top: "<postgres> Script-34", "<postgres> Week-4-Lab.sql", and "*<localhost> Script-36". The main editor area contains the following SQL code:

```

DROP TABLE IF EXISTS practica_oltp;

CREATE TABLE practica_oltp (
  id BIGINT AUTO INCREMENT PRIMARY KEY,
  nombre VARCHAR(100),
  saldo DECIMAL(10,2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

set session cte_max_recursion_depth = 10000000;

INSERT INTO practica_oltp (nombre, saldo)
WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 10000000
)
SELECT
  MD5(RAND()),           -- nombre
  ROUND(RAND()*10000, 2) -- saldo
FROM seq;

explain analyze
select * from clientes_mysql where id = 5000000;

```

Below the editor, a panel titled "Estadísticas 1" shows the execution statistics for the last query:

Name	Value
Updated Rows	10000000
Execute time	5m 5s
Start time	Tue Mar 03 00:23:48 CST 2026
Finish time	Tue Mar 03 00:28:54 CST 2026
Query	<pre> INSERT INTO practica_oltp (nombre, saldo) WITH RECURSIVE seq AS (SELECT 1 AS n UNION ALL SELECT n + 1 FROM seq WHERE n < 10000000) SELECT MD5(RAND()), -- nombre ROUND(RAND()*10000, 2) -- saldo FROM seq </pre>

```

--> CREATE TABLE practica_oltp (
  id INT(11) UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100),
  saldo DECIMAL(10,2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

set session cte_max_recursion_depth = 10000000;

-- INSERT INTO practica_oltp (nombre, saldo)
WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 10000000
)
SELECT
  MD5(RAND()),           -- nombre
  ROUND(RAND()*10000, 2) -- saldo
FROM seq;

SELECT COUNT(*) FROM practica_oltp;

-- EXPLAIN ANALYZE
SELECT * FROM practica_oltp WHERE id = 5000000;

-- Nota separada:
-- Single index lookup on clustered index (un solo B+Tree traversal)

```

Resultados 1

EXPLAIN ANALYZE SELECT * FROM practica_oltp WHERE id = 5000000

Grilla

	EXPLAIN
1	-> Rows fetched before execution (cost=0.0 rows=1) (actual time=0.00119..0.00128 rows=1 loops=1)¶

Texto

EXPERIMENTO 2

MySQL

The screenshot shows a MySQL IDE with four tabs: <postgres> Script-34, <postgres> Week-4-Lab.sql, *<postgres> Script-35, and *<localhost> Script-36. The active tab, Script-36, contains the following SQL code:

```
-- nooo esperauu.  
-- Single index lookup on clustered index (un solo B+Tree traversal)  
  
CREATE TABLE pk_bigint (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    data VARCHAR(100)  
) ENGINE=InnoDB;  
  
-- Tabla con PK UUID texto  
CREATE TABLE pk_uuid (  
    id CHAR(36) PRIMARY KEY,  
    data VARCHAR(100)  
) ENGINE=InnoDB;  
  
-- Insertar 5 millones  
INSERT INTO pk_bigint (data)  
WITH RECURSIVE seq AS (  
    SELECT 1 AS n  
    UNION ALL  
    SELECT n + 1 FROM seq WHERE n < 5000000  
)  
SELECT  
    MD5(RAND())  
FROM seq;  
  
INSERT INTO pk_uuid (id, data)  
WITH RECURSIVE seq AS (  
    SELECT 1 AS n
```

Below the SQL editor, the 'Estadísticas 1' (Statistics 1) window is open, displaying the following data:

Name	Value
Updated Rows	5000000
Execute time	2m 4s
Start time	Tue Mar 03 01:14:10 CST 2026
Finish time	Tue Mar 03 01:16:15 CST 2026
Query	INSERT INTO pk_bigint (data) WITH RECURSIVE seq AS (SELECT 1 AS n UNION ALL SELECT n + 1 FROM seq WHERE n < 5000000) SELECT MD5(RAND()) FROM seq

<postgres> Script-34 <postgres> Week-4-Lab.sql *<postgres> Script-35 *<localhost> Script-36 X

```
-- Tabla con PK UUID TEXT
CREATE TABLE pk_uuid (
  id CHAR(36) PRIMARY KEY,
  data VARCHAR(100)
) ENGINE=InnoDB;

-- Insertar 5 millones
INSERT INTO pk_bigint (data)
WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 5000000
)
SELECT
  MD5(RAND())
FROM seq;

INSERT INTO pk_uuid (id, data)
WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 5000000
)
SELECT
  UUID(),
  MD5(RAND())
FROM seq;

-- Tamaño en disco
```

Estadísticas 1 X	
Name	Value
Updated Rows	5000000
Execute time	2m 49s
Start time	Tue Mar 03 01:17:43 CST 2026
Finish time	Tue Mar 03 01:20:33 CST 2026
Query	INSERT INTO pk_uuid (id, data) WITH RECURSIVE seq AS (SELECT 1 AS n UNION ALL SELECT n + 1 FROM seq WHERE n < 5000000) SELECT UUID(), MD5(RAND()) FROM seq

<postgres> Script-34

<postgres> Week-4-Lab.sql

<postgres> Script-35

*<localhost> Script-36

```

SELECT 1 AS n
UNION ALL
SELECT n + 1 FROM seq WHERE n < 5000000
)
SELECT
    MD5(RAND())
FROM seq;

-- INSERT INTO pk_uuid (id, data)
WITH RECURSIVE seq AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM seq WHERE n < 5000000
)
SELECT
    UUID(),
    MD5(RAND())
FROM seq;

-- Tamaño en disco
SELECT table_name,
    round((data_length + index_length)/1024/1024,2) AS size_mb
FROM information_schema.tables
WHERE table_schema = DATABASE();

-- Fragmentación
SHOW TABLE STATUS LIKE 'pk_bigint';
SHOW TABLE STATUS LIKE 'pk_uuid';

```

TABLES 1

Grilla

Texto

Chart

1

2

3

A-Z TABLE_NAME

123 size_mb

pk_bigint

pk_uuid

practica_oltp

311,89

445

663

SELECT table_name, round((data_length + index_length)/1024/1024,2) AS size_mb

Enter a SQL expression to filter results (use Ctrl+Space)

Script-36

```
SELECT * AS n
UNION ALL
SELECT n + 1 FROM seq WHERE n < 5000000
)
SELECT
  MD5(RAND())
FROM seq;

-- INSERT INTO pk_uuid (id, data)
WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 5000000
)
SELECT
  UUID(),
  MD5(RAND())
FROM seq;

-- Tamaño en disco
SELECT table_name,
  round((data_length + index_length)/1024/1024,2) AS size_mb
FROM information_schema.tables
WHERE table_schema = DATABASE();

-- Fragmentación
SHOW TABLE STATUS LIKE 'pk_bigint';
SHOW TABLE STATUS LIKE 'pk_uuid';
```

tables(+) 1

SHOW TABLE STATUS LIKE 'pk_bigint'

	Name	Engine	Version	Row_format	Rows	Avg_row_length	Data_length	Max_data_length	Index_length	Data_free	Auto_increment	Create_time	
1	pk_bigint	InnoDB	10	Dynamic	4.855.235		67	327.041.024	0	0	6.291.456	5.046.196	2026-03-03

Script-36

```
SELECT * AS n
UNION ALL
SELECT n + 1 FROM seq WHERE n < 5000000
)
SELECT
  MD5(RAND())
FROM seq;

-- INSERT INTO pk_uuid (id, data)
WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 5000000
)
SELECT
  UUID(),
  MD5(RAND())
FROM seq;

-- Tamaño en disco
SELECT table_name,
  round((data_length + index_length)/1024/1024,2) AS size_mb
FROM information_schema.tables
WHERE table_schema = DATABASE();

-- Fragmentación
SHOW TABLE STATUS LIKE 'pk_bigint';
SHOW TABLE STATUS LIKE 'pk_uuid';
```

tables(+) 1

SHOW TABLE STATUS LIKE 'pk_uuid'

	Name	Engine	Version	Row_format	Rows	Avg_row_length	Data_length	Max_data_length	Index_length	Data_free	Auto_increment	Create_time	
1	pk_uuid	InnoDB	10	Dynamic	4.972.046		93	466.616.320	0	0	6.291.456	[NULL]	2026-03-03

EXPERIMENTO 3 POSTGRESQL

<postgres> Script-34

<postgres> Week-4-Lab.sql

*<postgres> Script-35 X

*<localhost> Script-36

```

-- heap fetch requires double access)

--Porque InnoDB usa índice clusterizado, mientras que PostgreSQL usa heap + índice separado.
--Cuando no hay caché, PostgreSQL necesita dos accesos a disco, InnoDB normalmente solo uno.

--EXPERIMENTO 3

DROP TABLE IF EXISTS mvcc_test;

CREATE TABLE mvcc_test (
    id BIGSERIAL PRIMARY KEY,
    saldo NUMERIC(10,2)
);

INSERT INTO mvcc_test (saldo)
SELECT random()*1000
FROM generate_series(1,10000000);

ANALYZE mvcc_test;

-- 1 millón de updates
UPDATE mvcc_test
SET saldo = saldo + 10
WHERE id <= 1000000;

-- Ver tuplas muertas
SELECT relname, n_live_tup, n_dead_tup
FROM pg_stat_user_tables

```

Estadísticas 1 X

Name	Value
Updated Rows	10000000
Execute time	1m 29s
Start time	Tue Mar 03 01:31:02 CST 2026
Finish time	Tue Mar 03 01:32:32 CST 2026
Query	INSERT INTO mvcc_test (saldo) SELECT random()*1000 FROM generate_series(1,10000000)

<postgres> Script-34 <postgres> Week-4-Lab.sql *<postgres> Script-35 X *<localhost> Script-36

```
-- heap fetch requerido (double acceso)

--Porque InnoDB usa índice clusterizado, mientras que PostgreSQL usa heap + índice separado.
--Cuando no hay caché, PostgreSQL necesita dos accesos a disco, InnoDB normalmente solo uno.

--EXPERIMENTO 3

DROP TABLE IF EXISTS mvcc_test;

CREATE TABLE mvcc_test (
    id BIGSERIAL PRIMARY KEY,
    saldo NUMERIC(10,2)
);

INSERT INTO mvcc_test (saldo)
SELECT random()*1000
FROM generate_series(1,1000000);

ANALYZE mvcc_test;

-- 1 millón de updates
UPDATE mvcc_test
SET saldo = saldo + 10
WHERE id <= 1000000;

-- Ver tuplas muertas
SELECT relname, n_live_tup, n_dead_tup
FROM pg_stat_user_tables
```

Estadísticas 1 X	
Name	Value
Updated Rows	1000000
Execute time	16s
Start time	Tue Mar 03 01:36:00 CST 2026
Finish time	Tue Mar 03 01:36:17 CST 2026
Query	UPDATE mvcc_test SET saldo = saldo + 10 WHERE id <= 1000000

<postgres> Script-34<postgres> Week-4-Lab.sql* <postgres> Script-35 X* <localhost> Script-36

▶

ⓘ

🔍

📄

AI

📄

⚠️

⚠️

⚙️

⋮

📄

📄

📄

📄

```
-- EXPERIMENTO 3

DROP TABLE IF EXISTS mvcc_test;

CREATE TABLE mvcc_test (
    id BIGSERIAL PRIMARY KEY,
    saldo NUMERIC(10,2)
);

INSERT INTO mvcc_test (saldo)
SELECT random()*1000
FROM generate_series(1,10000000);

ANALYZE mvcc_test;

-- 1 millón de updates
UPDATE mvcc_test
SET saldo = saldo + 10
WHERE id <= 1000000;

-- Ver tuplas muertas
SELECT relname, n_live_tup, n_dead_tup
FROM pg_stat_user_tables
WHERE relname = 'mvcc_test';

-- Tamaño antes de vacuum
SELECT pg_size_pretty(pg_total_relation_size('mvcc_test'));

VACUUM mvcc_test;
```

pg_stat_user_tables 1 X

SELECT relname, n_live_tup, n_dead_tup FROM pg_stat_user_tables; Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

1

A-Z relname

123 n_live_tup

123 n_dead_tup

1	mvcc_test	9.999.860	1.000.000	

Texto

<postgres> Script-34 <postgres> Week-4-Lab.sql * <postgres> Script-35 × * <localhost> Script-36

```
-- saldo NUMERIC(10,2)
);

INSERT INTO mvcc_test (saldo)
SELECT random()*1000
FROM generate_series(1,10000000);

ANALYZE mvcc_test;

-- 1 millón de updates
UPDATE mvcc_test
SET saldo = saldo + 10
WHERE id <= 1000000;

-- Ver tuplas muertas
SELECT relname, n_live_tup, n_dead_tup
FROM pg_stat_user_tables
WHERE relname = 'mvcc_test';

-- Tamaño antes de vacuum
SELECT pg_size_pretty(pg_total_relation_size('mvcc_test'));

VACUUM mvcc_test;

-- Tamaño después
SELECT pg_size_pretty(pg_total_relation_size('mvcc_test'));
```

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	1.788s
Start time	Tue Mar 03 01:39:53 CST 2026
Finish time	Tue Mar 03 01:39:55 CST 2026
Query	VACUUM mvcc_test

 <postgres> Script-34 <postgres> Week-4-Lab.sql *<postgres> Script-35 *<localhost> Script-36

```
SELECT random()*1000
```

```
SET saldo = saldo + 10
WHERE id <= 10000000;
```

```
FROM pg_stat_user_tables
WHERE relname = 'mvcc test';
```

```
SELECT pg_size_pretty(pg_total_relation_size( mvcc_test ));
```

Downloaded from <http://www.jstor.org/stable/2346282> on Tue, 20 Jun 2016 12:00:00 UTC
All use subject to [JSTOR Terms and Conditions](#)

```
-- Tamaño después
SELECT pg_size_pretty(pg_total_relation_size('mycc_test'));
```

pg_stat_user_tables 1 X

```
SELECT relname n live tup, n dead tup FROM pg_stat_user_tables;
```

© 2014 Pearson Education, Inc. or its affiliate(s). All rights reserved. This material is intended solely for the personal use of the individual user and is not to be disseminated broadly.

A-Z relname	123 n_live_tup	123 n_dead_tup
-------------	----------------	----------------

1	mvcc_test	9.999.717	0
---	-----------	-----------	---

Text	Text	Text	Text	Text

Category	Percentage
Very good	10%
Good	20%
Fair	30%
Poor	40%

EXPERIMENTO 4 PostgreSQL

The screenshot displays the PostgreSQL pgAdmin interface. The top pane shows a SQL script in a dark-themed editor. The script includes a table definition, an insert statement using a CASE statement for region selection and random data generation, and two queries for explanation and aggregation. The bottom pane shows the execution statistics for the first query.

```
local numeric,  
c1 TEXT, c2 TEXT, c3 TEXT, c4 TEXT, c5 TEXT,  
c6 TEXT, c7 TEXT, c8 TEXT, c9 TEXT  
);  
  
-- INSERT INTO ventas (region, producto, cliente, cantidad, precio, total)  
SELECT  
    CASE WHEN random() < 0.25 THEN 'Norte'  
         WHEN random() < 0.5 THEN 'Sur'  
         WHEN random() < 0.75 THEN 'Este'  
         ELSE 'Oeste' END,  
    md5(random()::text),  
    md5(random()::text),  
    (random()*10)::int,  
    random()*100,  
    random()*1000  
FROM generate_series(1,10000000);  
  
ANALYZE ventas;  
  
-- Query A  
EXPLAIN (ANALYZE, BUFFERS)  
SELECT * FROM ventas WHERE region = 'Norte';  
  
-- Query B  
EXPLAIN (ANALYZE, BUFFERS)  
SELECT SUM(total), COUNT(*)  
FROM ventas WHERE region = 'Norte';
```

Estadísticas 1

Name	Value
Updated Rows	10000000
Execute time	3m 26s
Start time	Tue Mar 03 01:49:36 CST 2026
Finish time	Tue Mar 03 01:53:02 CST 2026
Query	INSERT INTO ventas (region, producto, cliente, cantidad, precio, total) SELECT CASE WHEN random() < 0.25 THEN 'Norte' WHEN random() < 0.5 THEN 'Sur' WHEN random() < 0.75 THEN 'Este' ELSE 'Oeste' END, md5(random()::text), md5(random()::text), (random()*10)::int, random()*100, random()*1000 FROM generate_series(1,10000000)

```

<postgres> Script-34  <postgres> Week-4-Lab.sql  * <postgres> Script-35  * <localhost> Script-36

-- Query A
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ventas WHERE region = 'Norte';

-- Query B
EXPLAIN (ANALYZE, BUFFERS)
SELECT SUM(total), COUNT(*)
FROM ventas WHERE region = 'Norte';

-- Query C
EXPLAIN (ANALYZE, BUFFERS)
SELECT SUM(total), COUNT(*)
FROM ventas WHERE region = 'Norte';

```

Resultados 1	
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM ventas WHERE region = 'Norte';	
Enter a SQL expression to filter results (use Ctrl+Space)	
AZ QUERY PLAN	
1	Gather (cost=1000.00..199425.18 rows=14546 width=460) (actual time=17.446..30612.238 rows=2500013 loops=1)
2	Workers Planned: 2
3	Workers Launched: 2
4	Buffers: shared hit=14002 read=167817 dirtied=144020 written=141557
5	-> Parallel Seq Scan on ventas (cost=0.00..196970.58 rows=6061 width=460) (actual time=0.847..29344.858 rows=833338 loops=3)
6	Filter: (region = 'Norte'::text)
7	Rows Removed by Filter: 2499996
8	Buffers: shared hit=14002 read=167817 dirtied=144020 written=141557
9	Planning:
10	Buffers: shared hit=2 read=8
11	Planning Time: 73.876 ms
12	Execution Time: 30809.300 ms

Script-34 Script-35 Script-36

```

-- Query A
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ventas WHERE region = 'Norte';

-- Query B
EXPLAIN (ANALYZE, BUFFERS)
SELECT SUM(total), COUNT(*)
FROM ventas WHERE region = 'Norte';

```

Resultados 1

Enter a SQL expression to filter results (use Ctrl+Space)

Grilla Texto Chart Record

	A-Z QUERY PLAN
1	Finalize Aggregate (cost=198001.12..198001.13 rows=1 width=40) (actual time=12294.842..12311.503 rows=1 loops=1)
2	Buffers: shared hit=14098 read=167721
3	-> Gather (cost=198000.89..198001.10 rows=2 width=40) (actual time=12293.752..12310.738 rows=3 loops=1)
4	Workers Planned: 2
5	Workers Launched: 2
6	Buffers: shared hit=14098 read=167721
7	-> Partial Aggregate (cost=197000.89..197000.90 rows=1 width=40) (actual time=12173.830..12173.831 rows=1 loops=3)
8	Buffers: shared hit=14098 read=167721
9	-> Parallel Seq Scan on ventas (cost=0.00..196970.58 rows=6061 width=32) (actual time=1.589..11919.546 rows=833338 loop
10	Filter: (region = 'Norte'::text)
11	Rows Removed by Filter: 2499996
12	Buffers: shared hit=14098 read=167721
13	Planning:
14	Buffers: shared hit=12 read=6
15	Planning Time: 120.702 ms
16	Execution Time: 12335.411 ms

EXPERIMENTO 5 PostgreSQL

```

-- Script-34
md5(random()::text),
md5(random()::text),
(random()*10)::int,
random()*100,
random()*1000
FROM generate_series(1,10000000);

ANALYZE ventas;

-- Query A
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ventas WHERE region = 'Norte';

-- Query B
EXPLAIN (ANALYZE, BUFFERS)
SELECT SUM(total), COUNT(*)
FROM ventas WHERE region = 'Norte';

-- Al comparar Query A y Query B se observa que ambas consultas se ejecutan sobre un modelo row
--EXPERIMENTO 5

CREATE INDEX idx_ventas_covering
ON ventas(region)
INCLUDE (total);

```

Estadísticas 1	
Name	Value
Updated Rows	0
Execute time	31s
Start time	Tue Mar 03 02:08:53 CST 2026
Finish time	Tue Mar 03 02:09:25 CST 2026
Query	CREATE INDEX idx_ventas_covering ON ventas(region) INCLUDE (total)

<postgres> Script-34

<postgres> Week-4-Lab.sql

*<postgres> Script-35

*localhost> Script-36

```
md5(random()::text),
md5(random()::text),
(random()*10)::int,
random()*100,
random()*1000
FROM generate_series(1,10000000);

ANALYZE ventas;

-- Query A
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ventas WHERE region = 'Norte';

-- Query B
EXPLAIN (ANALYZE, BUFFERS)
SELECT SUM(total), COUNT(*)
FROM ventas WHERE region = 'Norte';

-- Al comparar Query A y Query B se observa que ambas consultas se ejecutan sobre un modelo row

--EXPERIMENTO 5

CREATE INDEX idx_ventas_covering
ON ventas(region)
INCLUDE (total);
```

Salida X

Enter a part of a message to search for here

la tabla «practica_oltp» no e
la tabla «mvcc_test» no exist
la tabla «ventas» no existe,

Resultados 1 X

EXPLAIN (ANALYZE, BUFFERS) SELECT SUM(total), COUNT(*) FROM

Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

Texto

Chart

Record

A-Z QUERY PLAN

1	Finalize Aggregate (cost=78845.79..78845.80 rows=1 width=40) (actual time=4276.026..4298.742 rows=1 loops=1)
2	Buffers: shared hit=18 read=12321 written=3458
3	-> Gather (cost=78845.56..78845.77 rows=2 width=40) (actual time=4273.790..4296.877 rows=3 loops=1)
4	Workers Planned: 2
5	Workers Launched: 2
6	Buffers: shared hit=18 read=12321 written=3458
7	-> Partial Aggregate (cost=77845.56..77845.57 rows=1 width=40) (actual time=4186.116..4186.121 rows=1 loops=3)
8	Buffers: shared hit=18 read=12321 written=3458
9	-> Parallel Index Only Scan using idx_ventas_covering on ventas (cost=0.56..72601.11 rows=1048889 width=11) (actual time=24.495..3965.333 rows=833338 loops=3)
10	Index Cond: (region = 'Norte'::text)
11	Heap Fetches: 12
12	Buffers: shared hit=18 read=12321 written=3458
13	Planning:
14	Buffers: shared hit=31 read=1
15	Planning Time: 22.314 ms
16	Execution Time: 4304.893 ms